

沈阳航空航天大学
计算机学院

实验报告

课程名称

操作系统

专 业

物联网工程

班 级

2234280102

学 号

223428010210

学生姓名

陈梓欣

指导教师

宗传玉

实验时间

周二 7-8 节

实验地点

工训楼 310

一、实验名称

Linux 系统内存管理

二、实验目的

1. 理解 Linux 系统段式、页式内存管理方式。
2. 理解 Linux 系统的二级页表结构及逻辑地址、线性地址、物理地址的映射过程。

三、实验内容和要求

1、编写代码实现页式管理分配物理页和释放物理页的过程

(1) 新建一个 Linux011 Kernel 项目添加一个系统调用内核函数 `int dump_physical_mem()`，该函数功能如下：

①在终端设备上显示当前内存信息，包括物理页总数、空闲页数量和占用页数量；

②调用 `get_free_page` 函数分配一个空闲的物理页，输出当前空闲页和占用页的数量；

③调用 `free_page` 函数将刚刚分配的物理页进行回收，最后输出当前空闲页和占用页的数量。

(2) 启动 Linux0.11 系统，使用 vi 编辑器编写程序 `main.c`，该程序调用 `dump_physical_mem()` 函数，程序执行结果如图 28 所示。


```

Page Count : 3840
Memory Count : 3840 * 4096 = 15728640 Byte

Free Page Count : 2968
Used Page Count : 872
*****After Allocate One Page*****
Allocate One Page : 0xFA6000
Free Page Count : 2967
Used Page Count : 873
*****After Free One Page*****
Free One Page : 0xFA6000
Free Page Count : 2968
Used Page Count : 872

```

图 1. 分配物理页和释放物理页的程序执行结果

实验详细内容请参考《Linux 内核实验教程_2.1》105-106 页

2. 编写代码输出页目录和页表信息

(1) 新建一个 Linux011 Kernel 项目添加一个系统调用内核函数，该系统调用的内核函数原型：int sys_table_mapping()。该函数首先调用/mm/memory.c 中的 calc_mem() 函数将内存中空闲页面的数量以及各个页表（从 0 号页起）映射的物理页数量输出到文本文件 a.txt 中，然后依次将下面信息输出到文本文件 a.txt 中。

①系统当前页目录信息

页目录信息格式为：Page Directory(页目录的物理页框号 | 所在线性地址)：

示例：Page Directory(PFN:0x0 | LA:0x00000000)

②每个页目录项 (PDE) 信息

页目录项 (PDE) 信息格式：

PDE: 下标 -> Page Table(页表的物理页框号 | 所在的线性地址)

示例：PDE: 0x0 -> Page Table(PFN:0x1 | LA:0x00001000)

③每个页目录项 (PDE) 对应的二级页表的页表项 (PTE) 信息

页表项 (PTE) 信息格式：

PTE: 下标 -> Physical Page(物理页框号 | 所在的线性地址)

示例：PTE: 0x0 -> Physical Page(PFN:0x0 | LA:0x00000000)

(2) 启动 Linux0.11 系统，使用 vi 编辑器编写程序 meminfo.c，该程序调用 sys_table_mapping() 函数。程序执行后生成文本文件 a.txt 示例如图 29 所示。


```

2968 pages free (of 3840)

Pg-dir[0] uses 1024 pages
Pg-dir[1] uses 1024 pages
Pg-dir[2] uses 1024 pages
Pg-dir[3] uses 1024 pages
Pg-dir[16] uses 160 pages
Pg-dir[32] uses 83 pages
Pg-dir[47] uses 2 pages
Pg-dir[48] uses 2 pages
Pg-dir[63] uses 1 pages
Pg-dir[64] uses 2 pages
Pg-dir[79] uses 1 pages
Page Directory(PFN:0x0 | LA:0x00000000)
    PDE: 0x0 -> Page Table(PFN:0x1 | LA:0x00001000)
        PTE: 0x0 -> Physical Page(PFN:0x0 | LA:0x00000000)
        PTE: 0x1 -> Physical Page(PFN:0x1 | LA:0x00001000)
        PTE: 0x2 -> Physical Page(PFN:0x2 | LA:0x00002000)
        PTE: 0x3 -> Physical Page(PFN:0x3 | LA:0x00003000)
        PTE: 0x4 -> Physical Page(PFN:0x4 | LA:0x00004000)

```

图 30. 文本文件 a.txt 示例

实验详细内容请参考《Linux 内核实验教程_2.1》116-117 页

四、实验设计

1. 进程与程序的区别

程序是指一个可执行文件，而进程是一个正在执行的程序实例。在 Linux 操作系统中，通过分时技术可以同时运行多个进程。分时技术的核心是将 CPU 的时间划分为短暂的时间片，分配给各个进程轮流运行。虽然某一时刻单核 CPU 上仅有一个进程运行，但由于时间片非常短（通常为几十毫秒），用户感觉多个进程在同时运行。

2. Linux 0.11 内核中的进程

- Linux 0.11 内核最多支持 64 个进程同时存在：
- 手工创建的第一个进程（init 进程）外，其余进程通过调用 fork 系统调用创建。
- 进程通过 进程标识号（PID） 区分。
- 每个进程由以下部分组成：

① 指令代码

② 数据区

③ 堆栈区

3. 进程的管理方式

(1) Linux 内核通过 进程表 对进程进行管理:

① 进程表 是一个数组, 定义在 kernel/sched.c 文件的第 112 行。

② 数组大小为 64, 每项为一个 task_struct 结构指针。

③ task_struct (任务数据结构) 又称为 进程控制块 (PCB) 或 进程描述符 (PD), 定义在 include/linux/sched.h 文件的第 146 行。

(2) 主要信息:

① 进程当前运行状态

② 信号

③ 进程号及父进程号

④ 运行时间累计值

⑤ 打开的文件描述符

⑥ 本任务的局部描述符表 (LDT)

⑦ 任务状态段 (TSS)

4. fork 系统调用创建新进程的过程

(1) 找到空闲的进程表项:

在进程数组中寻找未被使用的空项, 若无空位 (已达到 64 个进程), 则返回错误。

(2) 复制当前进程的 task_struct:

在主内存中为新进程申请一页, 复制当前进程的 task_struct。

将新进程的 state 设置为 不可中断的睡眠状态, 防止调度到未初始化的进程。

(3) 初始化新进程的 task_struct:

将当前进程设置为新进程的父进程。

清除信号位图, 重置统计数据。

设置初始运行时间片为 15。

设置 tss 各寄存器值：

tss.eax = 0 (子进程 fork 的返回值)

tss.esp0 指向新进程 task_struct 页面顶部

tss.ss0 设置为内核数据段选择符

若使用协处理器，则保存其状态至 tss.i387。

(4) 处理内存与页表：

为新进程设置代码段和数据段描述符。

复制父进程的页表，初始阶段与父进程共享物理页面。

当子进程尝试写入共享页面时，触发写时复制 (Copy on Write)，为其分配独立页面。

(5) 更新文件描述符：

父进程打开的文件，其引用计数加 1。

(6) 更新 GDT 描述符：

在全局描述符表 (GDT) 中设置新任务的 TSS 和 LDT 描述符。

(7) 设置任务状态：

将新任务状态设置为就绪态，返回新进程的 PID。

5. 写时复制 (Copy on Write) 技术

在进程创建时，子进程与父进程共享相同的物理内存页面，只有在任一进程尝试写入共享页面时，系统才实际分配独立的物理页面。这种技术有效减少了内存开销。

通过以上设计，实验旨在分析和验证 Linux 0.11 内核中 fork 系统调用的具体实现过程以及相关内核机制。

五、实验步骤及实验结果

1. 准备实验

- 1) 启动 Engintime Linux Lab。
- 2) 新建一个 Linux011 Kernel 实验项目。

2. 添加新的系统调用函数

为 Linux 0.11 添加 2 个新的系统调用函数，分别是：

① dump_physical_mem()

- a. 在终端设备上显示当前内存信息，包括物理页总数、空闲页数量和占用页数量；
- b. 调用 get_free_page 函数分配一个空闲的物理页，输出当前空闲页和占用页的数量；
- c. 调用 free_page 函数将刚刚分配的物理页进行回收，最后输出当前空闲页和占用页的数量。

② table_mapping()

- a. 该系统调用的内核函数原型：int sys_table_mapping()。
- b. 该函数首先调用/mm/memory.c 中的 calc_mem() 函数将内存中空闲页面的数量以及各个页表（从 0 号页起）映射的物理页数量输出到文本文件 a.txt 中，然后依次将下面信息输出到文本文件 a.txt 中。

3. 步骤如下：

- 1) 首先要为新系统调用分配一个唯一的系统调用号，以原来的最大系统调用号为基础加 1 作为新的系统调用号。在“项目管理器”窗口中双击打开 include/unistd.h 文件，在第 163-165 行添加新的系统调用号，如图 2 所示：

```
161 #define __NR_uselib 86
162
163 #define __NR_dump_physical_mem 87
164
165 #define __NR_table_mapping 88
166 // 以下定义系统调用嵌入式汇编函数。
```

图 3 添加新的系统调用号

- 2) 添加新系统调用号的同时，也要使系统调用总数在原来的基础上增加 1。打开 kernel/system_calls.s 文件，修改在第 73 行定义的系统调用总数，如下图：

```
73 nr_system_calls = 89 # Linux 0.11 版内核中的系统调用总数。
74
```

图 4 修改 Linux 内核的系统调用总数

- 3) 在 include/linux/sys.h 文件中的第 89-90 行使用 C 语言声明内核函数的原型，如图 4 所示。注意，这里定义的函数原型并不需要与函数的定义完全一致，只需要定义函数的返回值为 int 类型，参数为空，能够让标识符代表一个函数名称即可。


```

88
89 extern int sys_dump_physical_mem();
90 extern int sys_table_mapping();|

```

图 5 使用 C 语言声明内核函数的原型

在此文件的最后，向系统调用函数指针表 `sys_call_table[]` 中添加新系统调用函数的指针（注意，系统调用号必须与系统调用内核函数指针在系统调用函数表中的索引一一对应），如图 5 所示：

```

180 sys_use11b, //86
181 sys_dump_physical_mem, //87
182 sys_table_mapping //88
183 } .

```

图 6 在系统调用函数指针表中增加内核函数的指针

4) 在 `kernel/sys.c` 文件的最后编写代码，实现新系统调用对应的内核函数，如下图所示：

```

354 int sys_dump_physical_mem()
355 {
356     physical_mem();
357     return 0;
358 }

```

图 8 系统调用内核函数 `dump_physical_mem()`

5) `kernel.h` 文件中 22 行声明 `physical_mem()` 函数和 `fprintk(int fd, const char * fmt, ...)` 函数

```

21
22 int physical_mem();
23 int fprintk(int fd, const char * fmt, ...);|
24

```

图 9 `physical_mem()` 函数和 `fprintk` 函数声明


```

359 int sys_table_mapping()
360 {
361     unsigned long index_of_dir, index_of_table;
362     unsigned long entry;
363     unsigned long page_table_base;
364     unsigned long page_dir_base = 0;
365
366     __asm("cli");
367
368     calc_mem(); /*显示内存空闲页面数*/
369
370     //首先打印输出页目录信息, 格式为: PageDirectory(页目录的物理页框号 | 所在线性地址)
371     fprintf(1, "Page Directory(PFN:0x0 | LA:0x00000000)\n");
372
373     /*第一层循环, 遍历页目录中的所有PDE*/
374     for(index_of_dir = 0; index_of_dir < 1024; index_of_dir++)
375     {
376         entry = ((unsigned long*)page_dir_base)[index_of_dir];
377
378         if(!(entry & 1)) /*跳过无效的 PDE*/
379             continue;
380
381         //输出PDE信息, 格式为:
382         //PDE: 下标 -> Page Table(页表的物理页框号 | 所在线性地址)
383         fprintf(1, "\tPDE: 0x0 -> Page Table(PFN:0x1 | LA:0x00001000)\n", index_of_dir, entry >> 12, 0xfffff000 & entry);
384
385         /*页目录的高20位即为页表的物理地址, 页表的物理地址即为页表的逻辑地址*/
386         page_table_base = 0xfffff000 & entry;
387
388         /*第二层循环, 遍历页表中的所有PTE*/
389         for(index_of_table = 0; index_of_table < 1024; index_of_table++)
390         {
391             entry = ((unsigned long*)page_table_base)[index_of_table];
392
393             if(!(entry & 1)) /*跳过无效的PTE*/
394                 continue;
395
396             //输出PTE信息, 格式为:
397             //PTE: 下标 -> Physical Page(物理页框号 | 所在线性地址)
398             fprintf(1, "\t\tPTE: 0x0 -> Physical Page(PFN:0x0 | LA:0x00000000)\n", index_of_table, entry >> 12, (index_of_dir << 22) | (index_of_table << 12));
399         }
400     }
401     __asm("sti");
402     return 0;
403 }
404
405
406

```

图 9 系统调用内核函数 table_mapping()

6) 在 memory.c 文件中 782 行实现 int dump_physical_mem()函数、void calc_mem(void)函数

```

83
84 void calc_mem(void)
85 {
86     int i, j, k, free=0;
87     long * pg_tbl;
88
89     // 扫描内存页面映射数组mem_map[], 获取空闲页面数并显示。
90     for(i=0 ; i<PAGING_PAGES ; i++)
91         if (!mem_map[i]) free++;
92     fprintf(1, "%d pages free (of %d)\n\r", free, PAGING_PAGES);
93     // 扫描所有页目录项 (除0, 1 项), 如果页目录项有效, 则统计对应页表中有效页面数, 并显示。
94     for(i=0 ; i<1024 ; i++) {
95         if (l&pg_dir[i]) {
96             pg_tbl=(long *) (0xfffff000 & pg_dir[i]);
97             for(j=k=0 ; j<1024 ; j++)
98                 if (pg_tbl[j]&1)
99                     k++;
100             fprintf(1, "Pg-dir[%d] uses %d pages\n", i, k);
101         }
102     }
103 }

```

图 10 calc_mem(void)函数实现


```

805 int physical_mem() {
806     int i, free=0; /*free表示空闲页总数*/
807     unsigned long addr; /*分配的物理内存页地址*/
808
809     for(i=0;i<PAGING_PAGES;i++)
810     {
811         if(0 == mem_map[i])
812             {free++;}
813     }
814     printk("Page Count : %d\n", PAGING_PAGES);
815     //计算内存, kernel默认物理页为4K
816     printk("Memory Count : %d * 4096 = %d Byte\n", PAGING_PAGES, PAGING_PAGES * 4096);
817     printk("Free Page Count : %d\n", free);
818     printk("Used Page Count : %d\n", PAGING_PAGES - free);
819
820     //申请一个空闲页后的内存情况
821     printk("*****After Allocate One Page*****\n");
822     addr = get_free_page();
823     if(!addr) {
824         printk("get free page error!\n");
825         return 1;
826     }
827     printk("Allocate One Page : 0x%X\n", addr);
828     free = 0;
829     for(i=0;i<PAGING_PAGES;i++)
830     {
831         if(0 == mem_map[i]) //为0即mem_map[i]为空闲页
832             {free++;}
833     }
834     printk("Free Page Count : %d\n", free);
835     printk("Used Page Count : %d\n", PAGING_PAGES - free);
836
837     //释放掉申请的内存页后的内存情况
838     printk("*****After Release One Page*****\n");
839     free_page(addr);
840     printk("Free One Page : 0x%X\n", addr);
841     free = 0;
842     for(i=0;i<PAGING_PAGES;i++)
843     {
844         if(0 == mem_map[i]) //为0即mem_map[i]为空闲页
845         {
846             free++;
847         }
848     }
849     printk("Free Page Count : %d\n", free);
850     printk("Used Page Count : %d\n", PAGING_PAGES - free);
851
852     return 0;
853 }
854 }
855

```

图 11 dump_physical_mem()函数实现

7) printk.c 文件中第 23 到 27 行定义 fprintf, 第 55 行实现 fprintf。

```

23 #include<linux/sched.h>
24 // #include<sys/gstat.h>
25
26 static char buf[1024];
27 static char logbuf[1024];

```

图 12 fprintf 的声明


```

55 int fprintfk( int fd, const char * fmt, ... )
56 {
57     va_list args;
58     int i;
59     struct file * file;
60     struct m_inode * inode;
61     va_start (args, fmt);
62     i = vsprintf (logbuf, fmt, args);
63     va_end (args);
64
65     if( fd<3 )
66     {
67         __asm__ ("push %%fs\n\t"
68             "push %%ds\n\t"
69             "pop %%fs\n\t"
70             "pushl %0\n\t"
71             "pushl $_logbuf\n\t"
72             "pushl %1\n\t"
73             "call _sys_write\n\t"
74             "addl $8,%%esp\n\t"
75             "popl %0\n\t"
76             "pop %%fs"
77             ::"r" (i), "r" (fd):"ax", "dx");
78     }
79     else
80     {
81         if( !( file=task[0]->filp[fd] ) )
82             return 0;
83         inode=file->f_inode;
84         __asm__ ("push %%fs\n\t"
85             "push %%ds\n\t"
86             "pop %%fs\n\t"
87             "pushl %0\n\t"
88             "pushl $_logbuf\n\t"
89             "pushl %1\n\t"
90             "pushl %2\n\t"
91             "call _file_write\n\t"
92             "addl $12,%%esp\n\t"
93             "popl %0\n\t"
94             "pop %%fs"
95             ::"r" (i), "r" (file), "r" (inode) );
96     }
97     return i; // 返回字符串长度
98 }
99

```

图 13 fprintfk 的实现

8) 按 F7 生成 Linux 0.11 内核，修改语法错误直到生成成功。接下来需要在 Linux 0.11 中分别编写应用程序来测试新系统调用是否添加成功。

4. 在应用程序中测试新系统调用

- 1) 按 F5 启动调试。
- 2) 待 Linux 0.11 启动后，使用 vi 编辑器新建一个 main.c 文件。编辑 main.c 文件中的源代码(如图 7 所示)。其中，需要定义 `__LIBRARY__` 宏及包含 `unistd.h` 头文件，还需要再次定义 `__NR_dump_physical_mem` 宏，并使用 `_syscall0` 宏对系统调用函数进行定义，当该宏展开时，就会在源代码文件中使用 C 语言添加 `dump_physical_mem` 函数的完整实现。

```
[/usr/root]# cat main.c
#define __LIBRARY__
#include<unistd.h>
#define __NR_dump_physical_mem 87
_syscall0(int,dump_physical_mem)

int main(int argc,char **argv)
{
    dump_physical_mem();
    return 0;
}
[/usr/root]#
```

图 14 编辑 main.c 文件

- 3) 使用命令 `gcc main.c -o main` 生成可执行文件 main。

```
[/usr/root]# gcc main.c -o main
```

- 4) 执行 `sync` 命令，将文件保存到磁盘。

```
[/usr/root]# sync
[/usr/root]# sync main.c
sync[/usr/root]# sync
[/usr/root]#
```

图 15 将文件保存到磁盘

- 5) 执行 `chmod +x main` 命令为 main 文件添加可执行权限。

```
[/usr/root]# chmod +x main
[/usr/root]# ls -l main
-rwx--x--x 1 root root 9695 ??? ??  ??? main
[/usr/root]#
```

图 16 添加权限

- 6) 运行 main。如果 main 能输出正确的结果，则说明新系统调用添加成功。


```

[/usr/root]# main
Page Count : 3840
Memory Count : 3840 * 4096 = 15728640 Byte

Free Page Count : 2968
Used Page Count : 872
*****After Allocate One Page*****
Allocate One Page : 0xFA6000
Free Page Count : 2967
Used Page Count : 873
*****After Release One Page*****
Free One Page : 0xFA6000
Free Page Count : 2968
Used Page Count : 872
[/usr/root]#

```

图 17 正确输出分配物理页和释放物理页的程序执行结果

5. 仿照上述 2) -6) 步骤，测试新系统调用。

```

[/usr/root]# cat meminfo.c
#define __LIBRARY__
#include<unistd.h>
#define __NR_table_mapping 88
_syscall0( int, table_mapping )

int main( int argc, char **argv)
{
    table_mapping();
    return 0;
}
[/usr/root]#

```

图 18 编辑 meminfo.c 文件

6. 依次进行以下操作，将内存中空闲页面的数量以及各个页表（从 0 号页起）映射的物理页数量输出到文本文件 a.txt 中，然后依次将下面信息输出到文本文件 a.txt 中，并将该文件保存到软盘中。

```

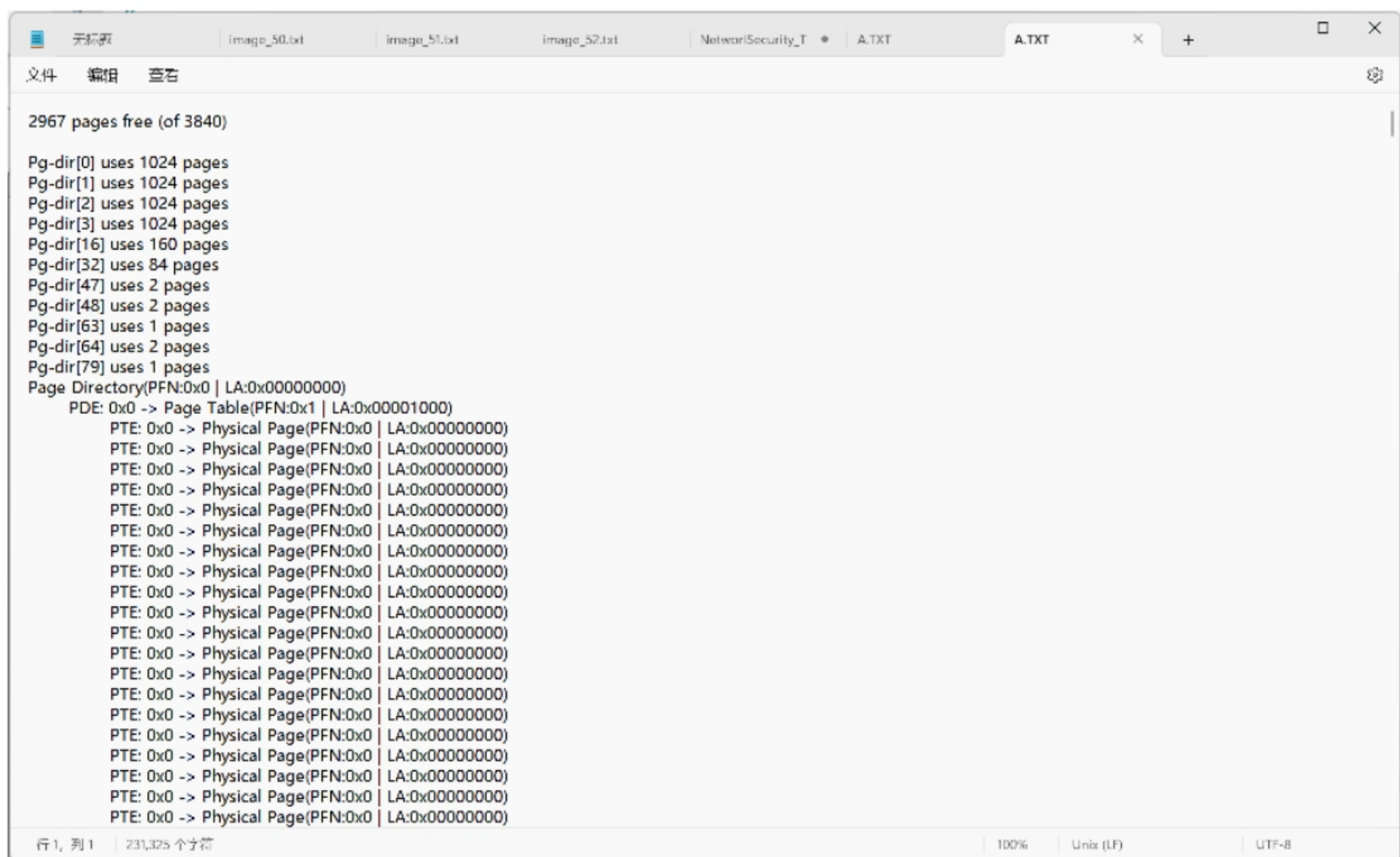
[/usr/root]# gcc meminfo.c -o meminfo
[/usr/root]# meminfo > a.txt
[/usr/root]# sync
[/usr/root]# sync a.txt
[/usr/root]# mcopy -n a.txt b:a.txt

```

图 19 a.txt 文件生成

7. 在左侧（项目管理器）最下面找到 floppyb.img, 双击打开

8. 查看 a.txt 文本文件内容。可以选择通过直接双击打开文件开始浏览或者复制到 Windows 再进行查看。



六、实验出现的问题及解决方法

解决方法：另外新建一个项目，重新开始。

七、结论

通过本次实验，我对 Linux 操作系统核心机制的理解得到了显著提升，尤其是在进程管理与调度方面的实现原理。本实验深入研究了 fork 系统调用的全过程，学习到进程创建不仅涉及当前进程数据的复制，还需要完成对任务描述符、内存空间和调度状态的初始化。特别是写时复制（Copy on Write）技术让我感受到操作系统在资源管理上的高效性，这种技术不仅避免了资源浪费，还提高了系统性能。

同时，通过对 task_struct 和进程表设计的分析，我认识到结构化数据在内核管理中的重要作用。进程表作为核心数据结构，不仅记录了系统中所有进程的状态，还为操作系统提供了统一的管理接口。任务控制块中的丰富信息，例如进程状态、寄存器值和时间片，为调度算法的实现奠定了坚实的基础。

在实验过程中，我还掌握了内核代码的阅读和分析方法，理解了如何从内核实现中探索操作系统的工作原理。这不仅提升了我的技术能力，也让我体会到操作系统设计的巧妙与复杂。本次实验让我对操作系统底层原理有了更加系统的认识，为后续深入学习提供了重要的理论与实践支持。

附录：程序源代码

//printk.c 文件

```
int fprintk( int fd, const char * fmt, ... )
{
    va_list args;
    int i;
    struct file * file;
    struct m_inode * inode;
    va_start (args, fmt);
    i = vsprintf (logbuf, fmt, args);
    va_end (args);

    if( fd<3 )
    {
        __asm__ ("push %%fs\n\t"
            "push %%ds\n\t"
            "pop %%fs\n\t"
            "pushl %0\n\t"
            "pushl $_logbuf\n\t"
            "pushl %1\n\t"
            "call _sys_write\n\t"
            "addl $8,%%esp\n\t"
            "popl %0\n\t"
            "pop %%fs"
            :: "r" (i), "r" (fd): "ax", "dx");
    }
    else
    {
        if( !( file=task[0]->filp[fd] ) )
            return 0;
        inode=file->f_inode;
        __asm__ ("push %%fs\n\t"
            "push %%ds\n\t"
            "pop %%fs\n\t"
            "pushl %0\n\t"
            "pushl $_logbuf\n\t"
            "pushl %1\n\t"
            "pushl %2\n\t"
            "call _file_write\n\t"
            "addl $12,%%esp\n\t"
            "popl %0\n\t"
```



```

        "pop %%fs"
        :: "r" (i), "r" (file), "r" (inode)    );
    }
    return i; // 返回字符串长度
}

```

//sys.c 文件

```

int sys_dump_physical_mem()
{
    physical_mem();
    return 0;
}

int sys_table_mapping()
{
    unsigned long index_of_dir,    index_of_table;
    unsigned long entry;
    unsigned long page_table_base;
    unsigned long page_dir_base = 0;

    __asm("cli");

    calc_mem();    /*显示内存空闲页面数*/

    //首先打印输出页目录信息，格式为：PageDirectory(页目录的物理页框号 | 所
    在线性地址)
    fprintf(1, "Page Directory(PFN:0x0 | LA:0x00000000)\n");

    /*第一层循环，遍历页目录中的所有 PDE*/
    for(index_of_dir = 0; index_of_dir < 1024; index_of_dir++)
    {
        entry = ((unsigned long*)page_dir_base)[index_of_dir];

        if(!(entry & 1)) /*跳过无效的 PDE*/
            continue;

        //输出 PDE 信息，格式为：
        //PDE: 下标 -> Page Table(页表的物理页框号 | 所在的线性地址)
        fprintf(1, "\tPDE: 0x0 -> Page Table(PFN:0x1 | LA:0x00001000)\n",
index_of_dir, entry >> 12, 0xfffff000 & entry);

        /*页目录的高 20 位即为页表的物理地址，页表的物理地址即为页表的逻辑地址*/
    }
}

```



```

page_table_base = 0xfffff000 & entry;

/*第二层循环，遍历页表中的所有 PTE*/
for(index_of_table = 0; index_of_table < 1024; index_of_table++)
{
    entry = ((unsigned long*)page_table_base)[index_of_table];

    if(!(entry & 1)) /*跳过无效的 PTE*/
        continue;

    //输出 PTE 信息，格式为：
    //PTE: 下标 -> Physical Page(物理页框号 | 所在的线性地址)
    fprintf(1, "\t\tPTE: 0x0 -> Physical Page(PFN:0x0 | LA:0x00000000)\n",
index_of_table, entry >> 12, (index_of_dir << 22) | (index_of_table << 12));
}
}
__asm("sti");

return 0;
}

```

//memory.c 文件

```

void calc_mem(void)
{
    int i,j,k,free=0;
    long * pg_tbl;

    // 扫描内存页面映射数组 mem_map[], 获取空闲页面数并显示。
    for(i=0; i<PAGING_PAGES; i++)
        if (!mem_map[i]) free++;
    fprintf(1, "%d pages free (of %d)\n", free, PAGING_PAGES);
    // 扫描所有页目录项 (除 0, 1 项)，如果页目录项有效，则统计对应页表中
    有效页面数，并显示。
    for(i=0; i<1024; i++) {
        if (1 & pg_dir[i]) {
            pg_tbl=(long *) (0xfffff000 & pg_dir[i]);
            for(j=k=0; j<1024; j++)
                if (pg_tbl[j] & 1)
                    k++;
            fprintf(1, "Pg-dir[%d] uses %d pages\n", i, k);
        }
    }
}

```



```

int physical_mem(){
    int i, free=0; /*free 表示空闲页总数*/
    unsigned long addr; /*分配的物理内存页地址*/

    for(i=0;i<PAGING_PAGES;i++)
    {
        if(0 == mem_map[i])
            {free++;}
    }
    printk("Page Count : %d\n", PAGING_PAGES);
    //计算内存, kernel 默认物理页为 4K
    printk("Memory Count : %d * 4096 = %d Byte\n\n", PAGING_PAGES,
PAGING_PAGES * 4096);
    printk("Free Page Count : %d\n", free);
    printk("Used Page Count : %d\n", PAGING_PAGES - free);

    //申请一个空闲页后的内存情况
    printk("*****After Allocate One Page*****\n");
    addr = get_free_page();
    if(!addr){
        printk("get free page error!\n");
        return 1;
    }
    printk("Allocate One Page : 0x%X\n", addr);
    free = 0;
    for(i=0;i<PAGING_PAGES;i++)
    {
        if(0 == mem_map[i]) //为 0 即 mem_map[i] 为空闲页
            {free++;}
    }
    printk("Free Page Count : %d\n", free);
    printk("Used Page Count : %d\n", PAGING_PAGES - free);

    //释放掉申请的内存页后的内存情况
    printk("*****After Release One Page*****\n");
    free_page(addr);
    printk("Free One Page : 0x%X\n", addr);
    free = 0;
    for(i=0;i<PAGING_PAGES;i++)
    {
        if(0 == mem_map[i]) //为 0 即 mem_map[i] 为空闲页
        {
            free++;
        }
    }
}

```



```
    }  
}  
printf("Free Page Count : %d\n", free);  
printf("Used Page Count : %d\n", PAGING_PAGES - free);  
  
return 0;  
  
}
```